



CarSeller

Documentation Technique & Guide Utilisateur

BTS SIO — 2024/2025

v1.0 — Juin 2026

Stack : AdonisJS 7 · Nuxt 4 · MySQL 8 · Docker

A. Documentation API · B. Documentation Web · C. Guide utilisateur

CarSeller — Documentation Technique et Utilisateur

Projet BTS SIO · 2024–2025

Stack : AdonisJS 7 (API) · Nuxt 4 (Web) · MySQL 8 · Docker

Sommaire

- [A. Documentation Technique — API](#)
 - [B. Documentation Technique — Web](#)
 - [C. Guide Utilisateur](#)
-

A. Documentation Technique — API

A.1 Présentation générale

L'API CarSeller est une API RESTful construite avec **AdonisJS v7** (framework Node.js TypeScript). Elle expose les données et la logique métier consommées par le frontend Nuxt. L'authentification repose sur des tokens Bearer stockés en base de données (`DbAccessTokensProvider`).

ÉLÉMENT	VALEUR
Framework	AdonisJS 7
Langage	TypeScript (ESM)
ORM	Lucid (AdonisJS)
Base de données	MySQL 8.0
Authentification	Bearer Token (opaque)
Port par défaut	3333
Validation	VineJS

A.2 Structure des dossiers

```
api/
├─ app/
│  ├─ controllers/      Contrôleurs HTTP (1 par ressource)
│  ├─ dto/              Data Transfer Objects (types TypeScript)
│  ├─ exceptions/       Gestionnaire global d'erreurs
│  ├─ factory/          Fabriques pour les seeders
│  ├─ middleware/        Middlewares (auth, JSON, CORS...)
│  ├─ models/           Modèles Lucid (entités BDD)
│  ├─ services/          Logique métier (1 service par domaine)
│  ├─ transformers/      Sérialisation des réponses JSON
│  └─ validators/        Schémas de validation des entrées
├─ config/              Configuration (auth, db, cors, session...)
├─ database/
│  ├─ migrations/        Migrations SQL
│  ├─ seeders/           Jeu de données de test
│  └─ schema.ts          Schémas auto-générés des modèles
├─ start/
│  ├─ routes.ts          Définition de toutes les routes
│  ├─ kernel.ts          Middlewares globaux
│  └─ validator.ts        Messages de validation personnalisés
└─ providers/           Providers AdonisJS
```

A.3 Modèles de données

User

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
firstname	string	Prénom
lastname	string	Nom
email	string	Adresse email (unique)
password	string	Hash bcrypt (non sérialisé)
phoneNumber	string \ null	Téléphone
type	string	customer ou technician
roles	any	Tableau JSON de rôles
createdAt	DateTime	Date de création
updatedAt	DateTime	Date de mise à jour

Vehicle (table unique, discriminée par type)

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
modelId	number \ null	FK → Model
customerId	number \ null	FK → User (propriétaire)
registration	string	Immatriculation
mileage	string	Kilométrage
year	number	Année
price	string \ null	Prix de vente
dailyPrice	string \ null	Tarif journalier location
type	string	salable , rentable , user_vehicle

Sous-types : SalableVehicle , RentableVehicle , UserVehicle héritent tous de Vehicle et pointent vers la même table vehicle .

Model

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
name	string	Nom du modèle
brandId	number \ null	FK → Brand
categoryId	number \ null	FK → Category

Brand

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
name	string	Nom de la marque

Category

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
name	string	Nom de la catégorie

Rental

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
userId	number	FK → User (locataire)
vehicleId	number	FK → Vehicle
startDate	Date	Début de location
endDate	Date	Fin de location
totalPrice	string	Prix total calculé

Ticket

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
customerId	number	FK → User (client)
technicianId	number \ null	FK → User (technicien assigné)
statusId	number	FK → TicketStatus
title	string	Titre
description	string	Description

TicketComment

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
ticketId	number	FK → Ticket
authorId	number	FK → User
comment	string	Texte du commentaire

TicketStatus

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
name	string	Ex: Ouvert , En cours , Résolu

MaintenanceRequest

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
customerId	number	FK → User (client)
technicianId	number	FK → User (technicien)
vehicleId	number	FK → Vehicle
typeId	number	FK → MaintenanceServiceType
requestDate	DateTime	Date souhaitée
approvedDate	DateTime \ null	Date approuvée

MaintenanceServiceType

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
name	string	Nom de la prestation
description	string	Description
duration	number	Durée (minutes)
price	string	Prix

Maintenance

CHAMP	TYPE	DESCRIPTION
id	number	Clé primaire
maintenanceRequestId	number	FK → MaintenanceRequest
customerId	number	FK → User
technicianId	number	FK → User
vehicleId	number	FK → Vehicle
typeId	number	FK → MaintenanceServiceType
maintenanceStatusId	number	FK → MaintenanceStatus
date	DateTime	Date planifiée

A.4 Routes et endpoints

Authentification — /auth

MÉTHODE	ROUTE	AUTH	DESCRIPTION
POST	/auth/signup	Non	Inscription
POST	/auth/login	Non	Connexion
POST	/auth/logout	Oui	Déconnexion

POST /auth/signup


```
// Body
{
  "firstname": "Jean",
  "lastname": "Dupont",
  "email": "jean@exemple.fr",
  "password": "motdepasse123",
  "passwordConfirmation": "motdepasse123"
}
// Réponse 201
{
  "data": {
    "user": { "id": 1, "firstname": "Jean", ... },
    "token": "oat_xxx..."
  }
}
```

POST /auth/login

```
// Body
{ "email": "jean@exemple.fr", "password": "motdepasse123" }
// Réponse 200
{
  "data": {
    "user": { "id": 1, ... },
    "token": "oat_xxx..."
  }
}
```

Compte — /account (auth requise)

MÉTHODE	ROUTE	DESCRIPTION
GET	/account/profile	Profil de l'utilisateur connecté
GET	/account/dashboard	Tableau de bord (véhicules, locations, RDV)
PUT	/account/profile	Mise à jour du profil

GET /account/dashboard — Réponse 200 :

```
{
  "data": {
    "user": { ... },
    "vehicles": [ { "id": 5, "registration": "AA-123-BB", ... } ],
    "rentals": [ { "id": 2, "startDate": "2025-06-01", ... } ],
    "appointments": [ { "id": 1, "subject": "Révision", "date": "2025-07-10", ... } ]
  }
}
```

MÉTHODE	ROUTE	AUTH	DESCRIPTION
GET	/salablevehicle	Non	Liste paginée avec filtres
GET	/salablevehicle/:vehicleId	Non	Détail d'un véhicule
POST	/salablevehicle/:vehicleId/purchase	Oui	Acheter un véhicule
POST	/salablevehicle	Non	Créer un véhicule (admin)
PUT	/salablevehicle/:vehicleId	Non	Modifier (admin)
DELETE	/salablevehicle/:vehicleId	Non	Supprimer (admin)

Paramètres de liste (query string) :

PARAMÈTRE	TYPE	DESCRIPTION
page	number	Numéro de page (défaut: 1)
limit	number	Éléments par page (défaut: 20)
min_price / max_price	number	Fourchette de prix
min_year / max_year	number	Fourchette d'année
min_mileage / max_mileage	number	Fourchette de kilométrage
brand_ids	string	IDs de marques séparés par virgule
model_ids	string	IDs de modèles
available	boolean	Uniquement les disponibles
order_by	string	price , year , mileage , created_at
order_dir	string	asc ou desc
preloads	string[]	Relations à charger (ex: model.brand)

Réponse paginée :

```
{
  "data": [ { "id": 1, "registration": "AB-123-CD", "price": 9500, ... } ],
  "meta": { "total": 45, "perPage": 12, "currentPage": 1, "lastPage": 4 }
}
```

MÉTHODE	ROUTE	AUTH	DESCRIPTION
GET	/rentablevehicle	Non	Liste paginée avec filtres
GET	/rentablevehicle/:vehicleId	Non	Détail + périodes réservées
GET	/rentablevehicle/:vehicleId/rentals	Non	Locations du véhicule
GET	/rentablevehicle/:vehicleId/availability	Non	Vérification de disponibilité
POST	/rentablevehicle/:vehicleId/rent	Oui	Réserver
POST	/rentablevehicle	Non	Créer (admin)
PUT	/rentablevehicle/:vehicleId	Non	Modifier (admin)
DELETE	/rentablevehicle/:vehicleId	Non	Supprimer (admin)

GET /rentablevehicle/:id/availability

```
?startDate=2025-07-01&endDate=2025-07-07
→ { "data": { "available": true } }
```

POST /rentablevehicle/:id/rent

```
// Body
{ "startDate": "2025-07-01", "endDate": "2025-07-07" }
// Réponse 201
{
  "data": { "id": 10, "startDate": "2025-07-01", "endDate": "2025-07-07", "totalPrice": 350, ... },
  "message": "Location confirmée"
}
```

Codes d'erreur métier :

CODE	HTTP	MESSAGE
VEHICLE_NOT_FOUND	404	Véhicule introuvable
VEHICLE_NOT_RENTABLE	400	Ce véhicule n'est pas louable
INVALID_DATES	422	Dates invalides
START_DATE_IN_PAST	422	La date de début ne peut pas être dans le passé
END_DATE_BEFORE_START	422	La date de fin doit être après la date de début
DATES_NOT_AVAILABLE	409	Le véhicule n'est pas disponible pour ces dates

Marques — /brand

MÉTHODE	ROUTE	AUTH	DESCRIPTION
GET	/brand	Non	Toutes les marques
GET	/brand/:brandId	Non	Détail d'une marque

Modèles — /model

MÉTHODE	ROUTE	AUTH	DESCRIPTION
GET	/model	Non	Tous les modèles
GET	/model/:modelId	Non	Détail d'un modèle

Demandes de maintenance — /maintenance-request (auth require)

MÉTHODE	ROUTE	DESCRIPTION
GET	/maintenance-request/types	Types de prestations disponibles
GET	/maintenance-request/technicians	Liste des techniciens
GET	/maintenance-request/vehicles	Véhicules du client connecté
GET	/maintenance-request	Liste des demandes
POST	/maintenance-request	Créer une demande
GET	/maintenance-request/:requestId	Détail d'une demande

POST /maintenance-request

```
// Body
{
  "typeId": 2,
  "vehicleId": 5,
  "technicianId": 3,
  "requestDate": "2025-07-15T10:00:00"
}
```

Codes d'erreur :

CODE	HTTP	MESSAGE
DATE_IN_PAST	422	La date doit être dans le futur
VEHICLE_NOT_FOUND	404	Véhicule introuvable ou non autorisé
TYPE_NOT_FOUND	404	Type de prestation introuvable
TECHNICIAN_NOT_FOUND	404	Technicien introuvable

Tickets — /ticket (auth requise)

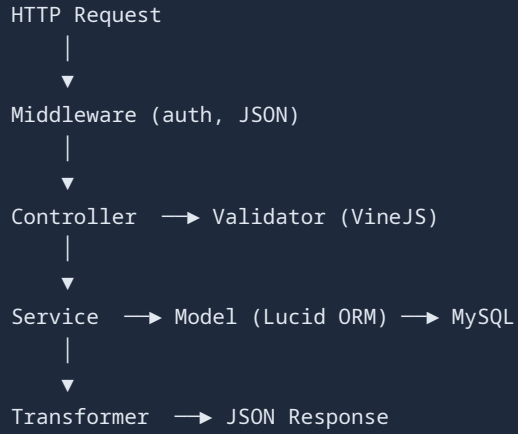
MÉTHODE	ROUTE	DESCRIPTION
GET	/ticket/statuses	Statuts disponibles
GET	/ticket	Liste des tickets de l'utilisateur
POST	/ticket	Créer un ticket
GET	/ticket/:ticketId	Détail + commentaires
POST	/ticket/:ticketId/comments	Ajouter un commentaire

Règle d'accès : Un client voit uniquement ses propres tickets. Un technicien voit les tickets qui lui sont assignés OU non assignés (*technicianId IS NULL*).

A.5 Middleware

MIDDLEWARE	RÔLE
auth_middleware	Vérifie le Bearer Token et injecte l'utilisateur
silent_auth_middleware	Tente l'auth sans bloquer la requête
force_json_response_middleware	Force le Content-Type <code>application/json</code>
container_bindings_middleware	Injecte les bindings IoC dans la requête

A.6 Architecture en couches



A.7 Authentification

L'API utilise des **opaque tokens** générés par `DbAccessTokensProvider`. Les tokens sont stockés dans la table `auth_access_tokens` (hash SHA-256).

Envoi du token :

```
Authorization: Bearer oat_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Cycle de vie :

1. Connexion → token créé en BDD
 2. Chaque requête → token décodé et vérifié
 3. Déconnexion → token supprimé de la BDD
-

B. Documentation Technique — Web

B.1 Présentation générale

Le frontend CarSeller est une application **Nuxt 4** (Vue 3, Composition API). Elle communique avec l'API via `$fetch` (offrant SSR/CSR transparent). L'état d'authentification est géré par **Pinia**.

ÉLÉMENT	VALEUR
Framework	Nuxt 4
UI Library	Vue 3 (Composition API)
State Management	Pinia
CSS	Tailwind CSS
Notifications	nuxt-notify
Icônes	@nuxt/icon
Port par défaut	3000

B.2 Structure des dossiers

```
web/
├─ app/
│  ├─ pages/           Pages Nuxt (routing automatique)
│  │  ├─ index.vue     Page d'accueil
│  │  ├─ auth/         Login, Register
│  │  ├─ ventes/       Catalogue vente
│  │  ├─ locations/    Catalogue location
│  │  ├─ compte/       Espace client
│  │  ├─ rendez-vous/  Demandes de maintenance
│  │  └─ tickets/      Tickets support
│  ├─ layouts/
│  │  └─ default.vue   Layout global (nav + footer)
│  ├─ stores/
│  │  └─ auth.ts        Store Pinia (session utilisateur)
│  ├─ services/        Couche API (wrappers $fetch)
│  ├─ composables/     Composables réutilisables
│  ├─ middleware/
│  │  └─ auth.ts        Middleware de protection des routes
│  ├─ plugins/
│  │  └─ auth.ts        Initialisation de session au démarrage
│  ├─ types/           Interfaces TypeScript
│  └─ utils/           Fonctions utilitaires (format, dates)
├─ assets/css/main.css  Styles globaux Tailwind
└─ public/             Assets statiques
```


B.3 Routing des pages

ROUTE	FICHIER	AUTH	DESCRIPTION
/	pages/index.vue	Non	Accueil / vitrine
/auth/login	pages/auth/login.vue	Non	Connexion
/auth/register	pages/auth/register.vue	Non	Inscription
/ventes	pages/ventes/index.vue	Non	Catalogue vente
/ventes/:id	pages/ventes/[id].vue	Non	Détail véhicule
/locations	pages/locations/index.vue	Non	Catalogue location
/locations/:id	pages/locations/[id].vue	Non	Réservation location
/compte	pages/compte/index.vue	Oui	Tableau de bord
/compte/modifier	pages/compte/modifier.vue	Oui	Modifier profil
/rendez-vous	pages/rendez-vous/index.vue	Oui	Liste des RDV
/rendez-vous/nouveau	pages/rendez-vous/nouveau.vue	Oui	Créer un RDV
/rendez-vous/:id	pages/rendez-vous/[id].vue	Oui	Détail RDV
/tickets	pages/tickets/index.vue	Oui	Liste des tickets
/tickets/nouveau	pages/tickets/nouveau.vue	Oui	Créer un ticket
/tickets/:id	pages/tickets/[id].vue	Oui	Détail ticket

B.4 Store Pinia — useAuthStore

Fichier : app/stores/auth.ts

État réactif :

PROPRIÉTÉ	TYPE	DESCRIPTION
<code>user</code>	<code>User \ null</code>	Données de l'utilisateur connecté
<code>token</code>	<code>string \ null</code>	Token Bearer (cookie persistant)
<code>loading</code>	<code>boolean</code>	Requête en cours
<code>initializing</code>	<code>boolean</code>	Initialisation de session
<code>error</code>	<code>string \ null</code>	Dernier message d'erreur
<code>isLoggedIn</code>	<code>computed<boolean></code>	<code>true</code> si token présent

Actions :

MÉTHODE	DESCRIPTION
<code>register(...)</code>	Inscription + connexion automatique
<code>login(email, password)</code>	Connexion via l'API
<code>logout()</code>	Suppression du token local + appel API
<code>me()</code>	Chargement du profil via <code>/account/profile</code>
<code>ensureInitialized()</code>	Garantit que <code>me()</code> n'est appelé qu'une fois
<code>setUser(user)</code>	Mise à jour locale de l'utilisateur
<code>clearSession()</code>	Effacement du token et de l'utilisateur

Persistence : Le token est stocké dans un cookie (`useCookie`) ce qui le rend accessible côté serveur (SSR) et survivre aux rechargements de page.

B.5 Middleware d'authentification

Fichier : `app/middleware/auth.ts`

Ce middleware est déclaré sur chaque page protégée via :

```
definePageMeta({ middleware: 'auth' })
```

Il attend la fin de l'initialisation du store auth (`ensureInitialized()`), puis redirige vers `/auth/login` si l'utilisateur n'est pas connecté.

B.6 Plugin d'initialisation

Fichier : `app/plugins/auth.ts`

Ce plugin s'exécute au démarrage de l'application (côté client uniquement). Il appelle `auth.ensureInitialized()` pour récupérer les données de l'utilisateur si un token valide est présent dans le cookie.

B.7 Services API

Chaque service est un wrapper autour de `$fetch` avec l'URL de base de l'API.

`services/api.ts` — Service de base

Fournit une instance préconfigurée de `$fetch` avec :

- `baseUrl` depuis `useRuntimeConfig().public.apiUrl`
- Header `Authorization: Bearer <token>` automatique

`services/salableVehicleService.ts`

MÉTHODE	DESCRIPTION
<code>list(params)</code>	Liste paginée avec filtres
<code>getById(id, preloads?)</code>	Détail d'un véhicule
<code>purchase(vehicleId)</code>	Acheter un véhicule

`services/rentableVehicleService.ts`

MÉTHODE	DESCRIPTION
<code>list(params)</code>	Liste paginée avec filtres
<code>getById(id)</code>	Détail + périodes réservées
<code>checkAvailability(id, startDate, endDate)</code>	Vérifier la disponibilité
<code>rent(vehicleId, startDate, endDate)</code>	Réserver

`services/brandService.ts`

MÉTHODE	DESCRIPTION
<code>getAll()</code>	Toutes les marques

`services/modelService.ts`

MÉTHODE	DESCRIPTION
<code>getAll(brandId?)</code>	Tous les modèles (ou filtrés par marque)

B.8 Composables

`composables/useAccountService.ts`

MÉTHODE	DESCRIPTION
<code>getDashboard()</code>	Charge le tableau de bord complet
<code>updateProfile(data)</code>	Met à jour le profil

`composables/useTicketService.ts`

MÉTHODE	DESCRIPTION
<code>getStatuses()</code>	Statuts de tickets
<code>list()</code>	Tickets de l'utilisateur
<code>getById(id)</code>	Détail + commentaires
<code>create(title, description)</code>	Créer un ticket
<code>addComment(ticketId, comment)</code>	Ajouter un commentaire

`composables/useMaintenanceRequestService.ts`

MÉTHODE	DESCRIPTION
<code>getServiceTypes()</code>	Types de prestations
<code>getTechnicians()</code>	Liste des techniciens
<code>getMyVehicles()</code>	Véhicules du client
<code>list()</code>	Demandes de l'utilisateur
<code>getById(id)</code>	Détail d'une demande
<code>create(data)</code>	Créer une demande

composables/usePaginatedList.ts

Composable générique pour gérer la pagination, le chargement et les erreurs d'une liste paginée.

B.9 Types TypeScript

FICHIER	INTERFACE
types/user.ts	User
types/account.ts	AccountDashboard
types/salableVehicle.ts	SalableVehicle
types/rentableVehicle.ts	RentableVehicle
types/userVehicle.ts	UserVehicle
types/rental.ts	Rental
types/ticket.ts	Ticket , TicketComment , TicketStatus
types/maintenanceRequest.ts	MaintenanceRequest , MaintenanceServiceType
types/api.ts	PaginatedResponse<T>

B.10 Utilitaires

utils/format.ts

FONCTION	DESCRIPTION
formatPrice(price)	Formatte en euros (fr-FR)
formatMileage(mileage)	Formatte en km (fr-FR)
formatDateTime(dt)	Date et heure lisibles
formatRole(role)	Libellé d'un rôle
formatUserType(type)	Libellé du type d'utilisateur

`utils/rentalDates.ts`

FONCTION	DESCRIPTION
<code>formatDateFr(date)</code>	Date au format français
<code>todayIso()</code>	Date du jour en ISO

B.11 Variables d'environnement

VARIABLE	DESCRIPTION
<code>NEXT_PUBLIC_API_URL</code>	URL de l'API (ex: <code>http://localhost:3333</code>)

C. Guide Utilisateur

C.1 Accès au site

Ouvrez votre navigateur et rendez-vous à l'adresse du site CarSeller.
Aucun compte n'est nécessaire pour consulter le catalogue de véhicules.

C.2 Créer un compte

1. Cliquez sur **"Créer un compte"** dans la barre de navigation ou sur la page d'accueil.
 2. Remplissez le formulaire :
 3. Prénom et Nom
 4. Adresse email
 5. Mot de passe (confirmé)
 6. Cliquez sur **"S'inscrire"**.
 7. Vous êtes automatiquement connecté et redirigé vers l'accueil.
-

C.3 Se connecter

1. Cliquez sur **"Se connecter"** dans la navigation.
 2. Saisissez votre adresse email et votre mot de passe.
 3. Cliquez sur **"Se connecter"**.
-

C.4 Parcourir les véhicules à vendre

1. Cliquez sur **"Voir les véhicules à vendre"** depuis l'accueil, ou sur **"Ventes"** dans la navigation.
2. La liste des véhicules disponibles s'affiche (12 par page).
3. **Filtrer** : utilisez la barre latérale gauche pour affiner :
4. Fourchette de prix
5. Kilométrage min/max
6. Année min/max
7. Marque (radio)

8. Modèle (radio, dépend de la marque sélectionnée)
 9. Critère de tri
 10. Cliquez sur **"Appliquer"** pour rafraîchir les résultats.
 11. Cliquez sur **"Réinitialiser"** pour effacer tous les filtres.
 12. Naviguez entre les pages avec les boutons **"Précédent"** / **"Suivant"**.
-

C.5 Acheter un véhicule

1. Depuis le catalogue des ventes, cliquez sur **"Détails"** pour un véhicule.
2. Consultez la fiche du véhicule (immatriculation, kilométrage, année, prix).
3. Cliquez sur **"Acheter"** (vous devez être connecté).
4. Un message de confirmation s'affiche et le véhicule apparaît dans votre compte.

Le véhicule devient indisponible à la vente une fois acheté.

C.6 Louer un véhicule

1. Cliquez sur **"Locations"** dans la navigation.
2. Parcourez le catalogue, filtrez si besoin.
3. Cliquez sur **"Louer"** pour un véhicule.
4. Sur la page de détail :
5. Consultez le tarif journalier et les périodes déjà réservées.
6. Sélectionnez une **date de début** et une **date de fin**.
7. Cliquez sur **"Vérifier la disponibilité"** puis **"Confirmer la location"**.
8. Le prix total est calculé automatiquement (jours × tarif journalier).

Vous devez être connecté pour louer. Les dates passées et déjà réservées sont bloquées.

C.7 Mon compte

1. Connecté, cliquez sur votre prénom ou **"Mon compte"** dans la navigation.
2. Le tableau de bord affiche :
3. Vos **informations personnelles**

4. Vos **véhicules achetés**
 5. Vos **locations** (avec statut : À venir / En cours / Terminée)
 6. Vos **rendez-vous** de maintenance
-

C.8 Modifier mon profil

1. Dans votre espace compte, cliquez sur "**Modifier mes informations**".
 2. Modifiez les champs souhaités :
 3. Prénom, Nom
 4. Email
 5. Téléphone
 6. Mot de passe (laissez vide pour ne pas le changer)
 7. Cliquez sur "**Enregistrer**".
-

C.9 Prendre un rendez-vous de maintenance

1. Allez dans "**Rendez-vous**" dans la navigation (connexion requise).
 2. Cliquez sur "**Nouveau rendez-vous**".
 3. Remplissez le formulaire :
 4. **Votre véhicule** : choisissez parmi vos véhicules achetés
 5. **Type de prestation** : révision, contrôle technique, etc.
 6. **Technicien** : sélectionnez un technicien disponible
 7. **Date et heure** souhaitées (doit être dans le futur)
 8. Cliquez sur "**Envoyer la demande**".
 9. La demande apparaît dans votre liste de rendez-vous avec le statut "**En attente**".
-

C.10 Tickets de support

Les tickets permettent de contacter l'équipe CarSeller pour toute question ou problème.

Créer un ticket

1. Allez dans "**Tickets**" dans la navigation.
 2. Cliquez sur "**Nouveau ticket**".
 3. Saisissez un **titre** et une **description** détaillée de votre demande.
-

4. Cliquez sur **"Envoyer"**.

Suivre et répondre à un ticket

1. Dans la liste des tickets, cliquez sur un ticket.
2. Consultez l'historique des échanges.
3. Ajoutez un **commentaire** dans le champ en bas et cliquez sur **"Envoyer"**.

Les techniciens voient les tickets qui leur sont assignés et peuvent répondre.

C.11 Se déconnecter

Cliquez sur **"Déconnexion"** dans la navigation. Votre session est supprimée côté serveur et côté client.
